Winston Industries LLC

Arithmetic Expression Evaluator in C++
Software Architecture Document

Version <1.1>

Revision History

Date	Version	Description	Author
<06/Apr/26>	<0.1>	Added Title and edited Introduction components	Davina Love
<07/Apr/26>	<0.11>	Changed architecture type to pipe-and-filter	Davina Love
<10/Apr/26>	<0.12>	Added changes to 4 and 4.1 of the Logical View section	Landrie Rolla
<11/Apr/26>	<0.13>	Added changes to 5, the Interface description	Christian Mitchell
<12/Apr/26>	<0.14>	Added changes to section 6. Added paragraph discussing benefits of modularity within pipe-and-filter framework including unintended behavior discussions and risks.	Conner Magee
<12/Apr/26>	<1.0>	Added section 3, completed ver. 1.0 of document	Davina Love
<4/<May/26>	<1.1>	Added UML Class Diagram	Davina Love

Table of Contents

Software Architecture Document

1. Introduction

The Software Architecture Document will outline the architecture of the Arithmetic Expression Evaluator in C. Below will define the purpose, scope, and various jargon used in the documentation.

1.1 Purpose

The purpose of this document is to provide a framework for designing the pipe and filter architecture required for the Arithmetic Expression Evaluator in C++.

The Software Architecture Document is intended for the eyes of *Winston Industries LLC* developers only. For them, this document should serve as a guide for how the program will be built from the ground up.

1.2 Scope

This document will apply to the general structure of the program ie: what the client facing will look like versus the backend class structure of the members and methods used. This document will not cover unit tests or any user manuals.

1.3 Definitions, Acronyms, and Abbreviations

Definitions

Class	Defined template that is used to create objects
Object	Unit that represents a real-world entity or instance of a class
Module	Small, interchangeable parts that make up a complex program
Pipe and filter	System that processes data streams
UI	What the user sees and can interact with
Tokenizer	Breaks down text into individual, identifiable pieces that will be used for input
Parser	Analyzes tokens and breaks them down in a format that a computer can understand
Evaluator	Performs user intended actions based on input from tokenizer and parser
Error Handler	Code that will catch and respond to errors

References

Date	*Document Title*	*Publishing Organization*
16/Feb/26	01-Project-Plan	Winston Industries LLC
14/Mar/26	02-Software-Requirements	Winston Industries LLC

Overview

This document will overview how the Arithmetic Expression Evaluator will be designed with a pipe-and-filter architecture approach consisting of five components: UI, tokenizer, parser, evaluator, and an error-handler. Each component will also include their own unit tests.

2. Architectural Representation

Architecture

The system utilizes a pipe and filter architecture where the pipe (input expression) has consecutive arithmetic and support filters (modules) that evaluate the expression and handle errors, based on the different functional requirements. This architecture allows for a mutable design where after the initial requirements are fulfilled additional filters can be added to the pipe to accommodate potentially new requirements.

Logical view

This design document contains a logical overview of the pipe and filter architecture used for the system. The Logical View displays the UI module, error handler module, and the order that the three filters apply to the input(pipe). These elements are titled based on their function and only show a high-level view of the flow of the system.

3. Architectural Goals and Constraints

The objective of the Arithmetic Expression Evaluator is to be lightweight and able to run on most machines. The user will only be providing input; the program will not expose any of the files on the user's machine.

Architecturally, the Arithmetic Expression Evaluator will employ object-oriented design by splitting each filter into its own module. Each module will contain its own methods and be able to be tested independently of the others.

This program is not designed for commercial use.

4. Logical View

The system follows a pipe-and-filter modular architecture that separates concerns into distinct components responsible for input handling, processing, and output. This will improve maintainability and testability by isolating functionality into different modules. Each design model will be compressed into the following:

- UI
    - Handles user interaction including input of expressions and displays results or error messages
- Filters
    - Holds core processing modules that will interpret and evaluate expressions, including tokenizing, evaluation, and parsing.
- Support
    - Provides shared serves such as validation, error detection, and utility functions used for all modules.

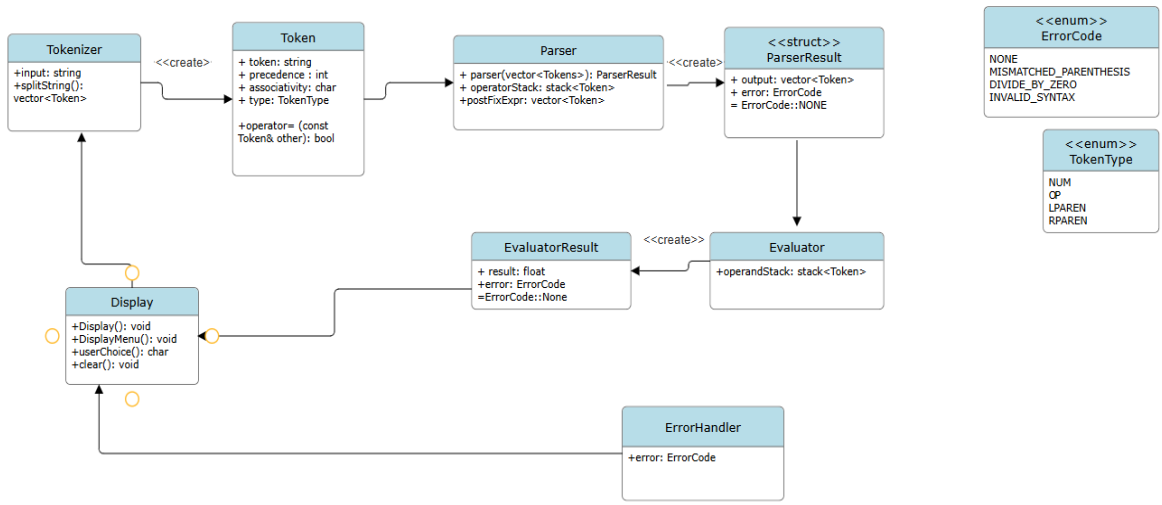
4.1 Overview

1. User Interface Module
    - Provides a user-friendly interface for input and output
    - Displays results and error messages

- Accepts user input (mathematical expressions)
- Performs basic validation (e.g. empty input, invalid characters)

2. Tokenizer Module
   - Breaks the input expression into tokens (operators, operands, parenthesis)
   - Converts raw strings into structured data for parsing
3. Parser Module
   - Analyzes tokes and determines execution order based on operator precedence
   - Handles parenthesis and builds a structured representation in RPN
4. Evaluator Module
   - Processes the structured output from the parser
   - Computes the resulting parsed expression following PEMDAS
5. Error Handling module
   - Communicates issues back to the UI module
   - Detects and manages errors that can include invalid syntax, mismatched parenthesis or division by zero.

4.2 Architecturally Significant Design Modules or Packages



UI	Display(), displayResult(), displayMenu(), userChoice(), clear()
Tokenizer	Token, SplitString(), vector<Token>
Parser	ParserResult, ParseTerm(), operatorStack stack<Token>
Evaluation	EvaluatorResult, Addition(), Subtraction(), Multiplication (), Division(), Exponents(), Modulo()
Error	error

Handler

4.2.1.1 UI

The display will have a Display() function that acts like a "monitor" to show that the program is running. The displayMenu() function is mostly cosmetic to show the main menu options. The userChoice() function acts like an input buffer, prompting the user for input and will throw an error if the input is invalid.

For invalid input, the display will show the error code, and then it will bring up the main menu options again for the user to put in a valid input.

4.2.2 Tokenizer

The tokenizer SplitString() method will split the string into individual components called Tokens. Each token will contain a string that acts as the 'value' of the token (example: "-4"), an integer that defines the precedence of the token, associativity (either 'L' or 'R') of the token, and a TokenType type to define if the token is an operator, number, or parenthesis.

The output of the tokenizer will be a vector of Token objects.

Whitespace will be ignored by the tokenizer, and invalid syntax found in the input will be caught here.

Operator precedence and associativity is defined in the table below.

**	3	R
*, /, %	2	L
+,-	1	L
(,)	0	NA

Negative numbers denoted with a unary minus (-) will be treated as one token.

4.2.3 Parser

The parser will take the vector<Token> as input and output a vector<Token> in Reverse Polish Notation (RPN). The parser will create a temporary stack of Tokens to hold operators and utilize the shunting yard algorithm to create a new vector of Tokens. This new vector will not have parenthesis as operator precedence is already handled in the output of the algorithm.
The resulting vector will be stored in a struct ParserResult that will also hold an ErrorCode member in the case there is an error to be handled.

4.2.4 Evaluator

The evaluator will examine the expression in RPN and continue the shunting yard algorithm to evaluate the expression, converting each Token into an integer and performing the required operations. The output will be a float.

This result will be stored in the struct EvaluatorResult. Similar to ParserResult, it will also hold an ErrorCode member.

4.2.5 Error Handler

The Error Handler will use the listed ErrorCodes and display an error message if an error is detected. For syntax errors, the error handler will also be able to pinpoint exactly where the error

Happened and provide feedback.

5. Interface Description

The UI will allow the user to input an expression. If the expression is deemed invalid at any point in the tokenizing, parsing, or evaluation process, an error will be returned back to the UI and user input will be queried again. Upon valid calculation, the UI will display the output of the expression.

The User may input any arithmetic expression composed of the following elements:

- Integer or floats (e.g. 2, 3.3, 258, 7.523)
- Arithmetic operators: +, -, *, /, %, or **
- Unary operators + and – applied to a numeric constant (e.g. (-3) + 3, +(3 + 3))
- Grouping with parentheses ()

Example Valid Input:

Please enter an expression: 4 * (3 + 2) % 7 - 1

Output: 5

Please enter an expression: ((5 * 2) - ((3 / 1) + ((4 % 3))))

Output: 6

Please enter an expression: +(-2) * (-3) – ((-4) / (+5))

Output: 6.8

Example Invalid Input:

Please enter an expression: 4 / 0

Error DivisionByZero, Division by zero is undefined

Please enter an expression: 2 * (4 + 3 – 1

Error UnmatchedParenthesis, this expression has unmatched opening and closing parentheses

Please enter an expression: ((7 * 3) ^ 2)

Error InvalidSyntax, this expression has an invalid arithmetic operator

6. Quality

The implementation of pipe-and-filter methodology extends both the modularity and extensibility of theprogram. Each filter can be individually modified and tested with sample outputs from the previous filter within the pipeline. This lack of interconnectivity between will make ease maintenance requirements and reduce the probability of unintentional bugged behavior by subsequently reducing the volume of changes needed for a modification. This is useful for all safety, security, and privacy concerns as minimal changes reduce the chances of unintended or vulnerable behavior within the software. The pipe-and-filter also creates an easily extensible program. In the event additional checks or features are required, all intermediate stages between filters are well-defined and as such additional filter programs can be inserted into the pipeline to fulfill the additional function so long as the output of the preceding and succeeding filters remain aligned. If this criterion is met, additional filters can be implemented in their entirety without modifying the existing filters.